

Déploiement d'une application conteneurisée sur un cluster Kubernetes

Mise en œuvre d'un cluster K3s sur une instance Amazon EC2

Nom : ECHIKR SAMIR

Classe : BTS SIO SISR

1. Introduction

Ce projet fait suite à la conteneurisation d'une application Python avec Docker. L'image produite ayant été publiée sur Docker Hub, l'objectif est désormais de la déployer sur un orchestrateur Kubernetes afin d'en assurer la disponibilité, la répartition de charge et la reprise automatique en cas de panne.

Le cluster est installé sur une instance Amazon EC2 à l'aide de K3s, une distribution allégée et certifiée de Kubernetes, particulièrement adaptée aux environnements à ressources limitées. L'application est déployée en trois exemplaires (pods) et exposée sur Internet.

2. Architecture mise en place

La chaîne complète est la suivante : le code source Python est stocké sur GitHub ; il est transformé en image Docker, elle-même publiée sur Docker Hub ; cette image est ensuite récupérée par le cluster Kubernetes hébergé sur une instance EC2, qui en exécute trois répliques accessibles depuis un navigateur.

L'application utilisée est un compteur de visites écrit en Python, écoutant sur le port 8000, dont l'image publique est echikr75/devops-app:latest.

3. Création de l'instance EC2

L'instance a été créée dans la région Europe (Paris) à partir d'une image Ubuntu Server 24.04 LTS, sur un type d'instance t3.small offrant 2 vCPU et 2 Go de mémoire, dimensionnement adapté à l'exécution d'un cluster K3s. Une paire de clés SSH nommée k8s-lab-key a été générée pour permettre la connexion sécurisée.

Lancer une instance [Informations](#)

Amazon EC2 vous permet de créer des machines virtuelles, ou des instances, qui s'exécutent sur le Cloud AWS. Démarrez rapidement en suivant les étapes simples indiquées ci-dessous.

Nom et balises [Informations](#)

Nom

 [Ajouter des balises supplémentaires](#)

Images d'applications et de systèmes d'exploitation (Amazon Machine Image) [Informations](#)

Une AMI contient le système d'exploitation, le serveur d'applications et les applications de votre instance. Si aucune AMI appropriée ne s'affiche ci-dessous, utilisez le champ de recherche ou choisissez [Parcourir d'autres AMI](#).

Q Effectuer une recherche dans notre catalogue complet, qui comprend des milliers d'images d'applications et de systèmes d'exploitation

Démarrage rapide

Amazon Linux

Ubuntu

Windows

Red Hat

SUSE Linux

Debian

[Explorer plus d'AMI](#)
Y compris les AMI d'AWS, de Marketplace et de la communauté

Amazon Machine Image (AMI)

Ubuntu Server 24.04 LTS (HVM), SSD Volume Type
ami-0e207c18bb303cc68 (64 bits (x86)) / ami-0d56693bd550e52c4 (64 bits (ARM))
Virtualisation: hvm ENA activé: true Type de périphérique racine: ebs

Éligible à l'offre gratuite

Description
Ubuntu Server 24.04 LTS (HVM),EBS General Purpose (SSD) Volume Type. Support available from Canonical: (<http://www.ubuntu.com/cloud/services>).
Canonical, Ubuntu, 24.04, amd64 noble image

Architecture	ID AMI	Date de publication	Nom d'utilisateur
64 bits (x86)	ami-0e207c18bb303cc68	2026-06-10	ubuntu

Fournisseur vérifié

Type d'instance [Informations](#) | [Obtenez des conseils](#)

Type d'instance

t3.small

Famille: t3 2 vCPU 2 Go Mémoire Génération actuelle: true

À la demande Windows base tarification: 0.042 USD par heure

À la demande Ubuntu Pro base tarification: 0.0271 USD par heure

À la demande RHEL base tarification: 0.0524 USD par heure

À la demande Linux base tarification: 0.0236 USD par heure

Éligible à l'offre gratuite

Toutes les générations

[Comparer les types d'instance](#)

Des frais supplémentaires s'appliquent pour les AMI avec un logiciel préinstallé

Paire de clés (connexion) [Informations](#)

Vous pouvez utiliser une paire de clés pour vous connecter en toute sécurité à votre instance. Assurez-vous d'avoir accès à la paire de clés sélectionnée avant de lancer l'instance.

Nom de la paire de clés - obligatoire

 [Créer une paire de clés](#)

Figure 1 — Configuration de l'instance EC2 (AMI Ubuntu, type t3.small, paire de clés)

[Alt+S]

☰ [EC2](#) > [Instances](#) > Lancer une instance

✓ **Succès**
Lancement de l'instance réussi (i-0d9e9cf29268b6e85)

Journal de lancement

Initialisation des requêtes	✓ Réussi
Création de groupes de sécurité	✓ Réussi
Création des règles de groupe de sécurité	✓ Réussi
Début du lancement	✓ Réussi

Figure 2 — Lancement de l'instance réussi

4. Configuration du pare-feu (groupe de sécurité)

Un groupe de sécurité agit comme un pare-feu virtuel : par défaut, tout le trafic entrant est bloqué. Deux règles ont donc été ajoutées : l'ouverture du port 22 pour l'administration en SSH, et celle du port 30080 destinée à exposer l'application déployée dans Kubernetes. Sans cette seconde règle, l'application serait injoignable depuis l'extérieur.

Paramètres réseau Informations

VPC - obligatoire Informations
vpc-09dab5b9fa008e10 (par défaut) ⓘ

Sous-réseau Informations
Aucune préférence ⓘ Créer un nouveau sous-réseau ⓘ

Zone de disponibilité Informations
Aucune préférence ⓘ Activer des zones supplémentaires ⓘ

Attribuer automatiquement l'adresse IP publique Informations
Activer ⓘ

Pare-feu (groupes de sécurité) Informations
Un groupe de sécurité est un ensemble de règles de pare-feu qui contrôlent le trafic de votre instance. Ajoutez des règles pour autoriser un trafic spécifique à atteindre votre instance.

☒ Créer un groupe de sécurité ☐ Sélectionner un groupe de sécurité existant

Nom du groupe de sécurité - obligatoire
k8s-lab-sg

Description - facultative Informations
launch-wizard-1 created 2026-07-23T10:34:39.984Z

Règles entrantes des groupes de sécurité

▼ Règle de groupe de sécurité 1 (TCP, 1433, 0.0.0.0/0) Supprimer

Type	Informations	Protocole	Informations	Plage de ports	Informations	Description - facultatif	Informations
MySQL		TCP		1433		par exemple, SSH pour le bureau de l'administrat	
Type de source	N'importe où	Source	Ajouter une adresse CIDR, une liste de préfixe				
			0.0.0.0/0				

▼ Règle de groupe de sécurité 2 (TCP, 22, 0.0.0.0/0) Supprimer

Type	Informations	Protocole	Informations	Plage de ports	Informations	Description - facultatif	Informations
ssh		TCP		22		par exemple, SSH pour le bureau de l'administrat	
Type de source	N'importe où	Source	Ajouter une adresse CIDR, une liste de préfixe				
			0.0.0.0/0				

▼ Règle de groupe de sécurité 3 (TCP, 30080, 0.0.0.0/0) Supprimer

Type	Informations	Protocole	Informations	Plage de ports	Informations	Description - facultatif	Informations
TCP personnalisé		TCP		30080		par exemple, SSH pour le bureau de l'administrat	
Type de source	N'importe où	Source	Ajouter une adresse CIDR, une liste de préfixe				
			0.0.0.0/0				

⚠ Les règles avec la source 0.0.0.0/0 autorisent toutes les adresses IP à accéder à votre instance. Nous vous recommandons de définir des règles de groupe de sécurité pour autoriser l'accès à partir d'adresses IP connues uniquement. ⓘ

Ajouter une règle de groupe de sécurité

Figure 3 — Règles entrantes du groupe de sécurité (ports 22 et 30080)

5. Connexion à l'instance

La connexion s'effectue en SSH depuis un poste Linux, en utilisant la clé privée téléchargée lors de la création de l'instance. Les droits du fichier de clé doivent être restreints, faute de quoi le client SSH refuse de l'utiliser.

```
chmod 400 k8s-lab-key.pem
ssh -i k8s-lab-key.pem ubuntu@13.39.150.35
```

```
The programs included with the Ubuntu system are free software;
the exact distribution terms for each program are described in the
individual files in /usr/share/doc/*/copyright.

Ubuntu comes with ABSOLUTELY NO WARRANTY, to the extent permitted by
applicable law.

To run a command as administrator (user "root"), use "sudo <command>".
See "man sudo_root" for details.

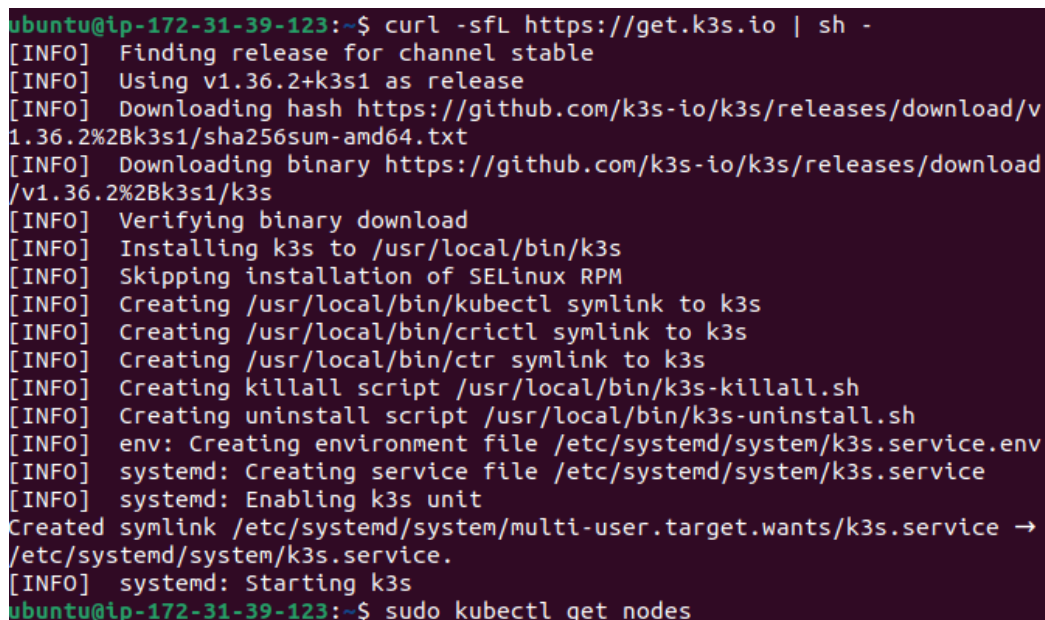
ubuntu@ip-172-31-39-123:~$
```

Figure 4 — Connexion SSH établie avec l'instance EC2

6. Installation du cluster K3s

K3s s'installe à l'aide d'un unique script fourni par l'éditeur. L'installation met en place le binaire k3s, l'outil en ligne de commande kubectl, ainsi qu'un service systemd assurant le démarrage automatique du cluster. L'opération dure moins d'une minute.

```
curl -sfL https://get.k3s.io | sh -  
sudo kubectl get nodes
```



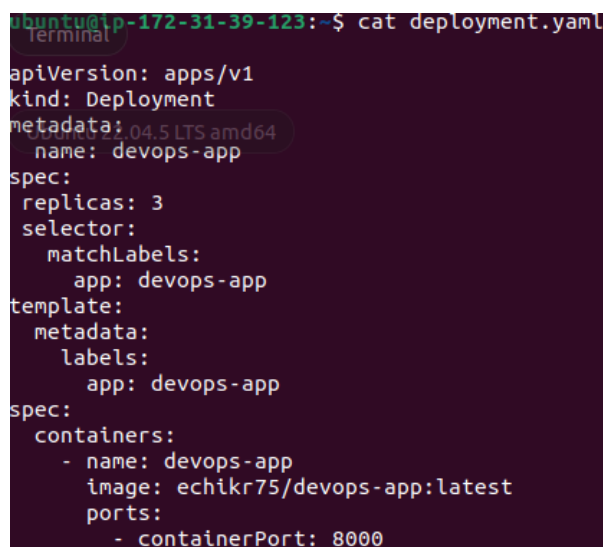
```
ubuntu@ip-172-31-39-123:~$ curl -sfL https://get.k3s.io | sh -  
[INFO] Finding release for channel stable  
[INFO] Using v1.36.2+k3s1 as release  
[INFO] Downloading hash https://github.com/k3s-io/k3s/releases/download/v1.36.2%2Bk3s1/sha256sum-amd64.txt  
[INFO] Downloading binary https://github.com/k3s-io/k3s/releases/download/v1.36.2%2Bk3s1/k3s  
[INFO] Verifying binary download  
[INFO] Installing k3s to /usr/local/bin/k3s  
[INFO] Skipping installation of SELinux RPM  
[INFO] Creating /usr/local/bin/kubectl symlink to k3s  
[INFO] Creating /usr/local/bin/crictl symlink to k3s  
[INFO] Creating /usr/local/bin/ctr symlink to k3s  
[INFO] Creating killall script /usr/local/bin/k3s-killall.sh  
[INFO] Creating uninstall script /usr/local/bin/k3s-uninstall.sh  
[INFO] env: Creating environment file /etc/systemd/system/k3s.service.env  
[INFO] systemd: Creating service file /etc/systemd/system/k3s.service  
[INFO] systemd: Enabling k3s unit  
Created symlink /etc/systemd/system/multi-user.target.wants/k3s.service → /etc/systemd/system/k3s.service.  
[INFO] systemd: Starting k3s  
ubuntu@ip-172-31-39-123:~$ sudo kubectl get nodes
```

Figure 5 — Installation de K3s et vérification du nœud

7. Rédaction des manifests Kubernetes

Kubernetes fonctionne de manière déclarative : plutôt que de décrire les opérations à effectuer, on décrit l'état souhaité du système, que l'orchestrateur se charge ensuite de maintenir. Deux fichiers au format YAML ont été rédigés.

Le Deployment définit le nombre de répliques souhaité (trois), l'image à utiliser et le port exposé par le conteneur. Le Service, de type NodePort, crée un point d'entrée unique vers ces trois pods et redirige le port 30080 de la machine vers le port 8000 de l'application.



```
ubuntu@ip-172-31-39-123:~$ cat deployment.yaml  
apiVersion: apps/v1  
kind: Deployment  
metadata:  
  name: devops-app  
spec:  
  replicas: 3  
  selector:  
    matchLabels:  
      app: devops-app  
  template:  
    metadata:  
      labels:  
        app: devops-app  
spec:  
  containers:  
    - name: devops-app  
      image: echikr75/devops-app:latest  
      ports:  
        - containerPort: 8000
```

Figure 6 — Contenu du fichier deployment.yaml

8. Déploiement de l'application

Les deux manifests sont appliqués au cluster. Kubernetes télécharge alors l'image depuis Docker Hub et crée les trois pods demandés.

```
sudo kubectl apply -f deployment.yaml
sudo kubectl apply -f service.yaml
```

```
ubuntu@ip-172-31-39-123:~$ sudo kubectl apply -f deployment.yaml
deployment.apps/devops-app created
ubuntu@ip-172-31-39-123:~$ sudo kubectl apply -f service.yaml
service/devops-app-service created
```

Figure 7 — Création du Deployment et du Service

```
sudo kubectl get pods
```

```
ubuntu@ip-172-31-39-123:~$ sudo kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
devops-app-6f964b5dd8-27nmd        1/1     Running   0           50s
devops-app-6f964b5dd8-2s9lg        1/1     Running   0           50s
devops-app-6f964b5dd8-cmhs4        1/1     Running   0           50s
```

Figure 8 — Les trois pods de l'application sont en état Running

```
sudo kubectl get svc
```

```
ubuntu@ip-172-31-39-123:~$ sudo kubectl get svc
NAME                TYPE        CLUSTER-IP    EXTERNAL-IP    PORT(S)          AGE
devops-app-service  NodePort    10.43.207.116 <none>         80:30080/TCP     73s
kubernetes          ClusterIP   10.43.0.1     <none>         443/TCP          15m
```

Figure 9 — Service NodePort exposant l'application sur le port 30080

9. Test de l'application

L'application est ensuite testée depuis un navigateur, en interrogeant l'adresse IP publique de l'instance sur le port 30080. Le compteur de visites répond correctement, ce qui valide l'ensemble de la chaîne : image publiée sur Docker Hub, récupérée par le cluster, exécutée dans les pods et exposée par le Service.

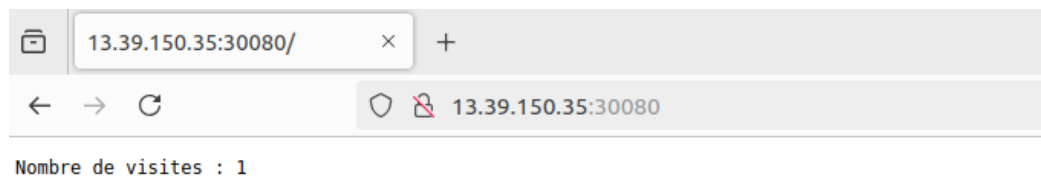


Figure 10 — Application accessible sur http://13.39.150.35:30080

10. Test de la reprise automatique (auto-réparation)

L'un des apports majeurs de Kubernetes est sa capacité à maintenir en permanence l'état déclaré. Pour le vérifier, l'un des pods a été volontairement supprimé.

```
sudo kubectl delete pod devops-app-6f964b5dd8-2s9lg
ubuntu@ip-172-31-39-123:~$ sudo kubectl delete pod devops-app-6f964b5dd8-2s9lg
pod "devops-app-6f964b5dd8-2s9lg" deleted from default namespace
```

Figure 11 — Suppression volontaire d'un pod

L'observation immédiate du cluster montre quatre pods : l'ancien en cours d'arrêt (Terminating) et un nouveau créé en quelques secondes seulement. Kubernetes n'attend pas la fin de l'arrêt pour lancer le remplaçant, ce qui garantit l'absence d'interruption de service. Le cluster revient ensuite automatiquement à trois pods, conformément à la valeur replicas déclarée.

```
ubuntu@ip-172-31-39-123:~$ sudo kubectl get pods
NAME                                READY   STATUS    RESTARTS   AGE
devops-app-6f964b5dd8-2s9lg         1/1     Terminating   0          7m32s
devops-app-6f964b5dd8-cmhs4         1/1     Running        0          7m32s
devops-app-6f964b5dd8-zj7mj        1/1     Running        0          8s
devops-app-6f964b5dd8-zq8vg        1/1     Running        0          2m22s
```

Figure 12 — Reprise automatique : le pod supprimé est immédiatement remplacé

11. Limite observée

En rafraîchissant plusieurs fois la page, le compteur de visites ne progresse pas de manière linéaire. Ce comportement s'explique par l'architecture retenue : le Service répartit les requêtes entre les trois pods, et chaque pod dispose de sa propre base SQLite interne, isolée des autres.

Cette observation illustre une problématique classique des applications distribuées : une base de données locale ne peut être partagée entre plusieurs répliques. En environnement de production, il conviendrait d'utiliser une base de données externe accessible en réseau (PostgreSQL, MySQL) ou un volume partagé, afin que l'ensemble des pods travaille sur les mêmes données.

12. Points d'attention et bonnes pratiques

- **Compatibilité entre image système et type d'instance** : la première tentative de lancement a échoué, l'AMI sélectionnée depuis le catalogue complet embarquant Microsoft SQL Server, logiciel non pris en charge sur un type d'instance t3.small. Toutes les AMI ne sont pas compatibles avec tous les gabarits : certaines images commerciales imposent un dimensionnement minimal. L'utilisation des images officielles proposées dans l'onglet « Démarrage rapide » évite cet écueil.
- **Cohérence entre le Service et le pare-feu** : un Service de type NodePort ouvre un port sur le nœud, mais celui-ci reste inaccessible tant que le groupe de sécurité AWS ne l'autorise pas explicitement. Les deux configurations, celle de Kubernetes et celle du fournisseur cloud, doivent être alignées ; c'est une source fréquente de dysfonctionnement lors d'un déploiement.
- **Sensibilité du format YAML** : la hiérarchie des manifests repose exclusivement sur l'indentation, les tabulations étant interdites. Une structure incorrecte est rejetée par l'API Kubernetes avec un message de type « unknown field ». Il est donc recommandé de valider les fichiers avant application, par exemple à l'aide de l'option `--dry-run`.
- **Libération des ressources** : l'instance EC2 a été résiliée à l'issue des tests. En environnement cloud, toute ressource laissée active continue d'être facturée, y compris inutilisée ; le suivi du cycle de vie des ressources fait partie intégrante du travail d'administration.

13. Conclusion

Ce projet a permis de mettre en œuvre une chaîne de déploiement complète, depuis le code source jusqu'à une application accessible sur Internet, orchestrée par Kubernetes et hébergée sur une infrastructure cloud. Les notions fondamentales de l'orchestration ont été abordées concrètement : déclaration d'un état souhaité, réplication, exposition réseau et reprise automatique sur incident.

Conformément aux bonnes pratiques de gestion des ressources cloud, l'instance EC2 a été résiliée à l'issue des tests afin de ne pas générer de coûts inutiles, les fichiers de configuration ayant préalablement été sauvegardés sur le dépôt GitHub du projet.